



ULTIMATE **C HANDBOOK**

By CodeWithHarry



PREFACE

Welcome to the "Ultimate C Programming Handbook," your comprehensive guide to mastering C programming. This handbook is designed for beginners and anyone looking to strengthen their foundational knowledge of C, a versatile and user-friendly programming language.

PURPOSE AND AUDIENCE

This handbook aims to make programming accessible and enjoyable for everyone. Whether you're a student new to coding, a professional seeking to enhance your skills, or an enthusiast exploring C, this handbook will definitely be helpful. C's simplicity and readability make it an ideal starting point for anyone interested in programming.

STRUCTURE AND CONTENT

The handbook is divided into clear, concise chapters, each focused on a specific aspect of C:

- **Fundamental Concepts:** Start with the basics and write your first program.
- **Practical Examples:** Illustrative examples and sample code demonstrate the application of concepts.
- **Hands-On Exercises:** End-of-chapter exercises reinforce learning and build confidence.

WHY C?

C is known for its efficiency and control, making it perfect for system-level programming. It is a low-level, compiled language that provides fine-grained control over hardware and memory, supporting applications in operating systems, embedded systems, game development, and high-performance computing. C's power and flexibility make it a valuable tool for both novice and experienced programmers looking to understand how computers work at a deeper level.

ACKNOWLEDGEMENTS

I extend my gratitude to the educators, programmers, and contributors who have shared their knowledge and insights, shaping the content of this handbook. Special thanks to all the students watching my content on YouTube and C community for maintaining a supportive and inspiring environment for learners worldwide.

CONCLUSION

Learning programming can be both exciting and challenging. The "Ultimate C Programming Handbook" aims to make your journey smooth and rewarding. Watch my video along with following this handbook for optimal learning. Let this guide be your stepping stone to success in the world of programming.

TABLE OF CONTENTS

PREFACE	1
Purpose and Audience	1
Structure and Content.....	1
Why C?.....	1
Acknowledgements	1
Conclusion	1
C Programming Handbook BY codewithharry	6
What is programming?	6
What is C?	6
Uses of C.....	6
Chapter 1: Variables, Constants & Keywords	7
Variables	7
Rules for naming variables in C	7
Constants	7
Types of constants	7
Keywords	8
Our first c program	8
Basic structure of a c program	9
Comments	9
Compilation and execution.....	9
Library functions	10
Types of variables	10
Receiving input from the user.....	10
Chapter 1- Practice Set.....	11
Chapter 2: Instructions and operators	12
Types of instructions	12
Type declaration instructions.....	12
Arithmetic instructions.....	12
Type conversion.....	13
Operator precedence in c	14
Operator precedence	14
Operator associativity	14
Control Instructions	15
Chapter 2 – Practice Set.....	16
Chapter 3: Conditional Instructions.....	17

Decision making instructions in c.....	17
if-else statement.....	17
Code example:	17
Relational operators in c.....	18
Logical operators	18
Usage of logical operators:	18
else if clause	18
Operator precedence	19
Conditional operators	19
Switch case control instruction.....	20
Chapter 3 – Practice Set.....	21
Chapter 4: Loop control instruction	22
Why loops	22
Types of loops.....	22
while loop.....	22
Increment and decrement operators.....	23
do-while loop	23
for loop.....	23
A case of decrementing for loop.....	24
The break statement in c	24
The continue statement in c.....	25
Chapter 4 – Practice Set.....	26
Project 1: Number guessing game	27
Chapter 5 – Functions and Recursion	28
What is a function?	28
Function prototype.....	28
Function call.....	28
Function definition	29
Important points	29
Types of functions	29
Why use functions	29
Passing values to function	29
Note:.....	30
Recursion	31
Important notes:	32
Chapter 5 – Practice set	33
Chapter 6- Pointers	34

The “address of” (&) operator	34
The ‘value at address’ operator (*).....	34
How to declare a pointer?.....	34
A program to demonstrate pointers.....	35
Output:	35
Pointer to a pointer.....	35
Types of function call	36
Call by value	36
Call by reference.....	36
Chapter 6 – Practice set	38
Chapter 7 – Arrays	39
Accessing elements	39
Initialization of an array	39
Arrays in memory	40
Pointer arithmetic	40
Accessing array using pointers.....	41
Passing array to functions	41
Multidimensional arrays	41
2-D arrays in memory	41
Chapter 7 – Practice Set.....	43
Chapter 8 – Strings	44
Initializing strings	44
Strings in memoryd	44
Printing strings.....	44
Taking string input from the user	44
gets() and puts().....	45
Declaring a string using pointers	45
Standard library functions for strings.....	45
strlen()	45
strcpy().....	46
strcat()	46
strcmp()	46
Chapter 8 – Practice Set.....	47
Chapter 9 – Structures	48
Why use Structures?	48
Array of structures	48
Initializing structures.....	49

Structures in memory	49
Pointer to structures	49
Arrow operator.....	49
Passing structure to a function	49
typedef keyword	50
Chapter 9 – Practice set	51
Chapter 10 – File I/O	52
File pointer	52
File opening modes in C	52
Types of files.....	53
Reading a file	53
Closing the file.....	53
Write to a file	53
fgetc() and fputc()	54
EOF : end of file.....	54
Chapter 10 – Practice Set.....	55
Project 2: Snake, Water, Gun.....	56
Chapter 11 – Dynamic Memory Allocation.....	57
Dynamic memory allocation	57
Function for Dma in C.....	57
malloc() function	57
calloc() function.....	57
free() function	58
realloc() function	58
Chapter 11 – Practice set	59

C PROGRAMMING HANDBOOK BY CODEWITHHARRY

WHAT IS PROGRAMMING?

Computer programming is a medium for us to communicate with computers. Just like we use 'Hindi' or 'English' to communicate with each other, programming is a way for us to deliver our instructions to the computer.

WHAT IS C?

C is a programming language.

C is one of the oldest and finest programming languages.

C was developed by Dennis Ritchie at AT&T's Bell labs, USA in 1972.

USES OF C

C language is used to program a wide variety of systems. Some of the uses of C are as follows:

1. Major parts of Windows, Linux and other operating systems are written in C.
2. C is used to write driver programs for devices like tablets, printers etc.
3. C language is used to program embedded systems where programs need to run faster in limited memory (Microwave, Cameras etc.)
4. C is used to develop games, an area where latency is very important, i.e., the computer must react quickly to user input.

INSTALLATION

We will use VS Code as our code editor to write our code and install MinGW gcc compiler to compile our C program.

Compilation is the process of translating high-level source code written in programming languages like C into machine code, which is the low-level code that a computer's CPU can execute directly. Machine code consists of binary instructions specific to a computer's architecture.

We can install VS Code and MinGW from their respective websites

Just install it like
a game!



CHAPTER 1: VARIABLES, CONSTANTS & KEYWORDS

VARIABLES

A variable is a container which stores a 'value'. In kitchen, we have containers storing Rice, Dal, Sugar etc. Similar to that, variables in C stores value of a constant.

Example:

```
a = 3;      // a is assigned "3"
b = 4.7;    // b is assigned "4.7"
c = 'A';    // c is assigned 'A'
```

RULES FOR NAMING VARIABLES IN C

1. First character must be an alphabet or underscore (_)
2. No commas, blanks are allowed.
3. No special symbol other than (_) allowed.
4. Variable names are case sensitive.

We must create meaningful variable names in our programs. This enhances readability of our programs.

CONSTANTS

An entity whose value does not change is called as a constant.

A variable is an entity whose value can be changed.

TYPES OF CONSTANTS

Primarily, there are three types of constants:

1. Integer Constant → 1,6,7,9
2. Real Constant → 322.1, 2.5, 7.0
3. Character Constant → 'a', '\$', '@' (must be enclosed within single quotes)

KEYWORDS

These are reserved words, whose meaning is already known to the compiler. There are 32 keywords available in C.

auto	double	int	struct
break	long	else	switch
case	return	enum	typedef
char	register	extern	union
const	short	float	unsigned
continue	signed	for	void
default	sizeof	goto	volatile
do	static	if	while

OUR FIRST C PROGRAM

```
#include <stdio.h>

int main() {
    printf("Hello, I am learning C with Harry");
    return 0;
}
```

BASIC STRUCTURE OF A C PROGRAM

All C programs must follow a basic structure. A C program starts with a main function and executes instructions present inside it.

Each instruction is terminated with a semicolon (;).

There are some rules which are applicable to all the C programs:

1. Every program's execution starts from main() function.
2. All the statements are terminated with a semicolon.
3. Instructions are case-sensitive.
4. Instructions are executed in the same order in which they are written.

COMMENTS

Comments are used to clarify something about the program in plain language. It is a way for us to add notes to our program. There are two types of comments in C.

1. Single line Comment: Single-line comments start with two forward slashes (//). Any information after the slashes // lying on the same line would be ignored (will not be executed).

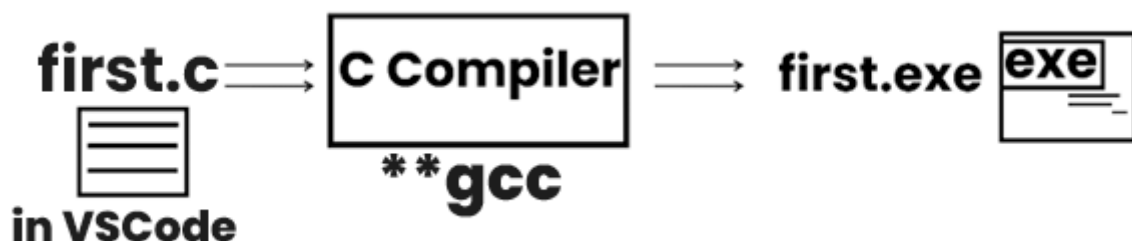
```
// This is a Single line comment.
```

2. Multi-line Comment: A multi-line comment starts with /* and ends with */. Any information between /* and */ will be ignored by the compiler.

```
/*  
This is a multi-line comment  
*/
```

Note: Comments in a C program are not executed and are ignored.

COMPILATION AND EXECUTION



A compiler is a computer program which converts a C program into machine language so that it can be easily understood by the computer.

A C program is written in plain text.

This plain text is combination of instructions in a particular sequence. The compiler performs some basic checks and finally converts the program into an executable.

LIBRARY FUNCTIONS

C language has a lot of valuable library functions which is used to carry out certain tasks. For instance printf() function is used to print values on the screen.

```
#include <stdio.h>
int main() {
    int i = 10;
    printf("This is %d\n", i);
    // %d for integers
    // %f for real values (floating-point numbers)
    // %c for characters
    return 0;
}
```

TYPES OF VARIABLES

1. Integer variables → `int a=3;`
2. Real variables → `int a=7; float a=7.7;`
3. Character variables → `char a= 'b';`

RECEIVING INPUT FROM THE USER

In order to take input from the user and assign it to a variable, we use scanf() function

Syntax:

```
scanf("%d", &i);
```

'&' is the "address of" operator and it means that the supplied value should be copied to the address which is indicated by variable i.

```
int= %d
float= %f
character= %c
```

CHAPTER 1- PRACTICE SET

1. Write a C program to calculate area of a rectangle:
 - a. Using hard coded inputs.
 - b. Using inputs supplied by the user.
2. Calculate the area of a circle and modify the same program to calculate the volume of a cylinder given its radius and height.
3. Write a program to convert Celsius (Centigrade degrees temperature to Fahrenheit).
4. Write a program to calculate simple interest for a set of values representing principal, number of years and rate of interest.

CHAPTER 2: INSTRUCTIONS AND OPERATORS

A C program is a set of instructions. Just like a recipe - which contains instructions to prepare a particular dish.

TYPES OF INSTRUCTIONS

1. Type declaration Instructions.
2. Arithmetic Instructions
3. Control Instructions.

TYPE DECLARATION INSTRUCTIONS

This is how you declare a variable in C

```
int a;  
float b;  
char c;
```

OTHER VARIATIONS:

Some other variations of this declaration look like this:

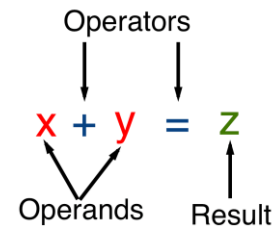
```
int a;      // Declare an integer variable 'a'  
float b;    // Declare a float variable 'b'  
int i = 10; // Declare and initialize 'i' with 10  
int j = i;  // Declare 'j' and initialize with 'i'  
int a = 2, b = 3, c = 4, d = 5; // Declare and initialize multiple variables  
  
int j1 = a + j - i; // Valid: use previously defined variables  
  
// Invalid: 'a' is used before declaration  
// float b = a + 3;  
// float a = 1.1;  
  
// Valid: Assigning the same value to multiple variables  
int a, b, c, d;  
a = b = c = d = 30; // a, b, c, d all equal to 30
```

ARITHMETIC INSTRUCTIONS

Arithmetic instructions perform mathematical operations.

Here are some of the commonly used operators in C language:

- + (Addition)
- - (Subtraction)
- * (Multiplication)
- / (Division)
- % (Modulus)



Note:

1. Operands can be int/float etc. + - * / are arithmetic operators.

```
int b = 2, c = 3;
int z; z = b*c; //legal
int z; b*c = z; //illegal (not allowed)
```

2. % is the modular division operator
 - % → returns the remainder
 - % → cannot be applied on float
 - % → sign is same as of numerator (-5%2=-1)
3. No operator is assumed to be present.

```
int i = ab // invalid
int i = a * b //valid
```

4. There is no operator to perform exponentiation in C however we can use pow (x,y) from <math.h> (more later).

TYPE CONVERSION

An Arithmetic operation between

- int and int → int
- int and float → float
- float and float → float

Example:

- 5/2 becomes 2 as both the operands are int
- 5.0/2 becomes 2.5 as one of the operands is float
- 2/5 becomes 0 as both the operands are int

NOTE:

In programming, type compatibility is crucial. For `int a = 3.5;`, the float 3.5 is demoted to 3, losing the fractional part because a is an integer. Conversely, for `float a = 8;`, the integer 8 is promoted to 8.0, matching the float type of a and retaining precision.

```
int a = 3.5; // In this case 3.5 (float) will be demoted to 3 (int)
             // because a is not able to store floats.

float a = 8; // a will store 8.0 | 8 -> 8.0 (promotion to float)
```

Quick Quiz: `int k = 3.0 / 9;` value of k? and why?

Ans: $3.0/9 = 0.333$. But since k is an int, it cannot store floats & value 0.33 is demoted to 0.

OPERATOR PRECEDENCE IN C

Have a look at the below statement:

$3*x - 8*y$ is $(3x)-(8y)$ or $3(x-8y)$?

In C language simple mathematical rules like BODMAS, no longer apply.

The answer to the above questions is provided by operator precedence & associativity.

OPERATOR PRECEDENCE

The following table lists the operator priority in C

Priority	Operators
1 st	* / %
2 nd	+ -
3 rd	=

Operators of higher priority are evaluated first in the absence of parenthesis.

OPERATOR ASSOCIATIVITY

When operators of equal priority are present in an expression, the tie is taken care of by associativity.

$x*y/z \rightarrow (x*y)/z$

$x/y*z \rightarrow (x/y)*z$

*, / follows left to right associativity

Pro Tip: Always use parenthesis in case of confusion

CONTROL INSTRUCTIONS

Determines the flow of control in a program four types of control instructions in C are:

1. Sequence Control instructions.
2. Decision Control instructions
3. Loop Control instructions
4. Case Control instructions.

CodeWithHarry

CHAPTER 2 – PRACTICE SET

1. Which of the following is invalid in C?
 - a. `int a=1; int b = a;`
 - b. `int v = 3*3;`
 - c. `char dt = '21 dec 2020';`
2. What data type will `3.0/8 - 2` return?
3. Write a program to check whether a number is divisible by 97 or not.
4. Explain step by step evaluation of $3*x/y - z+k$, where $x=2$, $y=3$, $z=3$, $k=1$
5. `3.0 + 1` will be:
 - a. Integer.
 - b. Floating point number.
 - c. Character.

CHAPTER 3: CONDITIONAL INSTRUCTIONS

Sometimes we want to watch comedy videos on YouTube if the day is Sunday.

Sometimes we order junk food if it is our friend's birthday in the hostel.

You might want to buy an umbrella if it's raining, and you have the money.

You order the meal if dal or your favourite bhindi is listed on the menu.

All these are decisions which depends on a condition being met.

In C language too, we must be able to execute instructions on a condition(s) being met.

DECISION MAKING INSTRUCTIONS IN C

- if-else statement
- switch statement

IF-ELSE STATEMENT

The syntax of an if-else statement in C looks like:

```
if (condition_to_be_checked) {  
    // Statements if condition is true  
} else {  
    // Statements if condition is false  
}
```

CODE EXAMPLE:

```
int a = 23;  
if (a > 18)  
{  
    printf("you can drive \n");  
}
```

Note that else block is not necessary but optional.

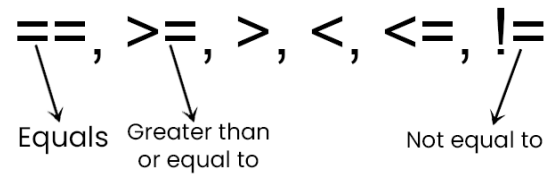
RELATIONAL OPERATORS IN C

Relational operators are used to evaluate conditions (true or false) inside the if statements.

Some examples of relational operators are:

`==, >=, >, <, <=, !=`

Important note: '=' is used for assignment whereas '==' is used for equality check.



The condition can be any valid expression. In C a non-zero value is considered to be true.

LOGICAL OPERATORS

`&&`, `||` and `!`, are three logical operators in C. These are read as “AND”, “OR” and “NOT”

They are used to provide logic to our C programs.

USAGE OF LOGICAL OPERATORS:

1. `&&` (AND) → is true when both the conditions are true
 - a. “1 and 0” is evaluated as false.
 - b. “0 and 0” is evaluated as false.
 - c. “1 and 1” is evaluated as true.
2. `||` (OR) → is true when at least one of the conditions is true. (1 or 0 → 1) (1 or 1 → 1)
3. `!` (NOT) → returns true if given false and false if given true
 - a. `!(3==3)` → evaluates to false
 - b. `!(3>30)` → evaluates to true.

As the number of conditions increases, the level of indentation increases. This reduces readability. Logical operators come to rescue in such cases.

ELSE IF CLAUSE

Instead of using multiple if statements, we can also use else if along with it thus forming an if-else if-else ladder.

CODE EXAMPLE

A typical if - else if - else ladder look like this:

```
if{
    // Statements
}
else if{
    // Statements
}
else{
    // Statements
}
```

IMPORTANT NOTE

1. Using if-else if -else reduces indents.
2. The last “else” is optional.
3. Also there can be any number of “else if”.
4. Last else is executed only if all conditions fail.

OPERATOR PRECEDENCE

Priority	Operator
1 st	!
2 nd	*, /, %
3 rd	+, -
4 th	<>, <=, >=
5 th	==, !=
6 th	&&
7 th	
8 th	=

CONDITIONAL OPERATORS

A shorthand “if – else” can be written using the conditional or ternary operators

```
condition ? expression-if-true : expression-if-false
// Here "?" and ":" are called Ternary Operators
```

SWITCH CASE CONTROL INSTRUCTION

switch-case is used when we have to make a choice between number of alternatives for a given variable.

```
switch (integer expression)
{
    case c1:
        // code;

    case c2:
        // code;           // c1, c2 & c3 -> Constants
                           // code -> Any valid C code.

    case c3:
        // code;

    default:
        // code;
}
```

The value of integer-expression is matched against c1, c2, c3... If it matches any of these cases, that case along with all subsequent “case” and “default” statements are executed.

Quick Quiz: Write a program to find grade of a student given his marks based on below:

90 – 100 => A
80 – 90 => B
70 – 80 => C
60 – 70 => D
50 – 60 => E
<50 => F

Some Important Notes:

- We can use switch-case statements even by writing cases in any order of our choice (not necessarily ascending).
- char values are allowed as they can be easily evaluated to an integer.
- A switch can occur within another but in practice this is rarely done.

CHAPTER 3 – PRACTICE SET

1. What will be the output of this program

```
int a = 10;
if (a = 11)
    printf("I am 11");
else
    printf("I am not 11");
```

2. Write a program to determine whether a student has passed or failed. To pass, a student requires a total of 40% and at least 33% in each subject. Assume there are three subjects and take the marks as input from the user.
3. Calculate income tax paid by an employee to the government as per the slabs mentioned below:

Income Slab	Tax
2.5 – 5.0L	5%
5.0L - 10.0L	20%
Above 10.0L	30%

Note that there is no tax below 2.5L. Take income amount as an input from the user.

4. Write a program to find whether a year entered by the user is a leap year or not.
Take year as an input from the user.
5. Write a program to determine whether a character entered by the user is lowercase or not.
6. Write a program to find greatest of four numbers entered by the user.

CHAPTER 4: LOOP CONTROL INSTRUCTION

WHY LOOPS

Sometimes we want our programs to execute few sets of instructions over and over again. For example: Printing 1 to 100, first 100 even numbers etc.

Hence loops make it easy for a programmer to tell computer that a given set of instructions must be executed repeatedly.

TYPES OF LOOPS

Primarily there are three types of loops in C language:

1. while loop
2. do-while loop
3. for loop

We will look into these one by one:

WHILE LOOP

```
while (condition is true) {  
    // Code  
    // The block keeps executing as long as the condition is true  
}
```

Example:

```
int i = 0;  
while (i < 10) {  
    printf("the value of i is %d\n", i);  
    i++;  
}
```

Note: If the condition never becomes false, the while loop keeps getting executed. Such loop is known as an infinite loop.

Quick Quiz: Write a program to print natural numbers from 10 to 20 when initial loop counter is initialized to 0.

The loop counter need not be int, it can be float as well.

INCREMENT AND DECREMENT OPERATORS

`i++` → `i` is increased by 1

`i--` → `i` is decreased by 1

```
// Decrement i first and then print
printf("--i = %d\n", --i);

// Print i first and then decrement
printf("i-- = %d\n", i--);
```

- `+++` operator does not exist.
- `i+=2` is compound assignment which translates to `i = i + 2`
- Similar to `+=` operator we have other operators like `-=`, `*=`, `/=`, `%=`.

DO-WHILE LOOP

The syntax of do-while loop looks like this:

```
do {
    //code;
} while (condition);
```

The do-while loop works very similar to while loop.

- 'while' checks the condition & then executes the code.
- 'do-while' executes the code & then checks the condition.

In simpler terms we can say:

do-while loop = while loop which executes at least once.

Quick Quiz: Write a program to print first 'n' natural number using do-while loop.

Input: 4

Output: 1
2
3
4

FOR LOOP

The syntax of a typical 'for' loop looks like this:

```
for (initialize; test; increment or decrement)
{
```



```
//code;  
}
```

- Initialize → Setting a loop counter to an initial value.
- Test → Checking a condition.
- Increment → Updating the loop counter.

Example:

```
for (i=0; i<3; i++){  
    printf("%d\n", i);  
    printf("\n");  
}  
  
// Output:  
//      0  
//      1  
//      2
```

Quick Quiz: Write a program to print first 'n' natural numbers using for loop

A CASE OF DECREMENTING FOR LOOP

```
for (i=5; i ; i--)  
    printf("%d\n", i);
```

This for loop will keep on running until i become 0.

The loop runs in following steps:

1. 'i' is initialized to 5.
2. The condition "i" (0 or none) is tested.
3. The code is executed.
4. 'i' is decremented.
5. Condition 'i' is checked & code is executed if it's not 0.
6. And so on until 'i' is non 0.

Quick Quiz: Write a program to print 'n' natural numbers in reverse order.

THE BREAK STATEMENT IN C

The 'break' statement is used to exit the loop irrespective of whether the condition is true or false.

Whenever a "break" is encountered inside the loop, the control is sent outside the loop

Let us see this with the help of an example:

```

for (i=0; i<1000; i++){
    printf("%d\n",i);
    if (i==5){
        break;
    }
}

```

OUTPUT

```

0
1
2
3
4
5

```

The output of the above program will be below (and not 0 to 100)

THE CONTINUE STATEMENT IN C

The 'continue' statement is used to immediately move to the next iteration of the loop.

The control is taken to the next iteration thus skipping everything below "continue" inside the loop for that iteration.

Example:

```

#include <stdio.h>

int main() {
    int skip = 5;
    int i = 0;
    while (i < 10) {
        if (i == skip) {
            i++;
            continue; // skips the rest of the loop body for i == 5
        }
        printf("%d\n", i);
        i++;
    }
    return 0;
}

```

Notes:

1. Sometimes, the name of the variable might not indicate the behaviour of the program.
2. 'break' statement completely exits the loop.
1. 'continue' statement skips the particular iteration of the loop.

CHAPTER 4 – PRACTICE SET

1. Write a program to print multiplication table of a given number n.
2. Write a program to print multiplication table of 10 in reversed order.
3. A do while loop is executed:
 - a. At least once.
 - b. At least twice.
 - c. At most once.
4. What can be done using one type of loop can also be done using the other two types of loops – true or false?
5. Write a program to sum first ten natural numbers using while loop.
6. Write a program to implement program 5 using 'for' and 'do-while' loop.
7. Write a program to calculate the sum of the numbers occurring in the multiplication table of 8. (consider 8 x 1 to 8 x 10).
8. Write a program to calculate the factorial of a given number using a for loop.
9. Repeat 8 using while loop.
10. Write a program to check whether a given number is prime or not using loops.
11. Implement 10 using other types of loops.

PROJECT 1: NUMBER GUESSING GAME

We will write a program that generates a random number and asks the player to guess it. If the player's guess is higher than the actual number, the program displays "Lower number please". Similarly, if the user's guess is too low, the program prints "Higher number please".

When the user guesses the correct number, the program displays the number of guesses the player used to arrive at the number.

Hint: Use loop & use a random number generator.

CodeWithHarry

CHAPTER 5 – FUNCTIONS AND RECURSION

Sometimes our program gets bigger in size and it's not possible for a programmer to track which piece of code is doing what.

Function is a way to break our code into chunks so that it is possible for a programmer to reuse them.

WHAT IS A FUNCTION?

A function is a block of code which performs a particular task.

A function can be reused by the programmer in a given program any number of times.

Syntax:

```
#include <stdio.h>

// Function prototype
void display();

int main() {
    int a; // Variable declaration
    display(); // Function call
    return 0; // Return statement
}

// Function definition
void display() {
    printf("hi i am display\n"); // Printing the message
}
```

FUNCTION PROTOTYPE

A function prototype informs the compiler about a function that will be defined later in the program.

The void keyword indicates that the function does not return any value.

FUNCTION CALL

A function call instructs the compiler to execute the function's body when the call is made.

Note that program execution starts from the main function and follows the sequence of instructions written.

FUNCTION DEFINITION

This part contains the exact set of instructions executed during the function call.

When a function is called from `main()`, the main function pauses and temporarily suspends. During this time, control transfers to the called function. Once the function finishes executing, `main()` resumes.

Quick Quiz: Write a program with three functions

1. Good morning function which prints “good morning”.
2. Good afternoon function which prints “good afternoon”.
3. Good night function which prints “good night”.

main() should call all of these in order 1→2→3

IMPORTANT POINTS

- Execution of a C program starts from `main()`.
- A C program can have more than one function.
- Every function gets called directly or indirectly from `main()`.

TYPES OF FUNCTIONS

There are two functions in C. Let's talk about them.

1. Library functions → Commonly required functions grouped together in a library file on disk.
2. User defined function → These are the functions declared and defined by the user.

WHY USE FUNCTIONS

1. To avoid rewriting the same logic again and again.
2. To keep track of what we are doing in a program
3. To test and check logic independently.

PASSING VALUES TO FUNCTION

We can pass values to a function and can get a value in return from a function.

Have a look at the code snippet below:

```
int sum (int a, int b)
```

A function prototype in programming is a declaration of a function that specifies its name, return type, and parameters (if any) but does not include the function body.

The above prototype means that sum is a function which takes values 'a' (of type int) and 'b' (of type int) and returns a value of type int.

Function definition of sum can be:

```
int sum (int a, int b) { // a and b are parameters
    int c;
    c = a+b;
    return c;
}
// Now we can call sum (2,3); from main to get 5 in return. Here 2 & 3 are arguments.
int d = sum (2,3); // d becomes 5
```

NOTE:

1. Parameters are the values or variable placeholders in the function definition. Example a & b.
2. Arguments are the actual values passed to the function to make a call. Example 2 & 3.
3. A function can return only one value at a time.
4. If the passed variable is changed inside the function, the function call doesn't change the value in the calling function.

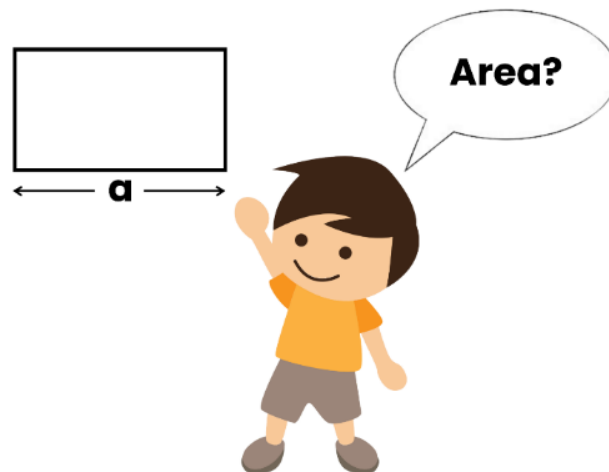
```
int change(int a) {
    a = 77; // Misnomer
    return 0;
}
```

'change' is a function which pretends to change 'a' to 77. Now if we call it from main like this

```
int b=22;
change(b); // The value of b remains 22
printf("b is %d", b); // Prints "b is 22"
```

This happens because a copy of 'b' is passed to the change function

Quick Quiz: Use the library function to calculate the area of a square with side a.



RECURSION

A function defined in C can call itself. This is called recursion. A function calling itself is also called 'recursive' function.

Example:

A very good example of recursion is factorial.

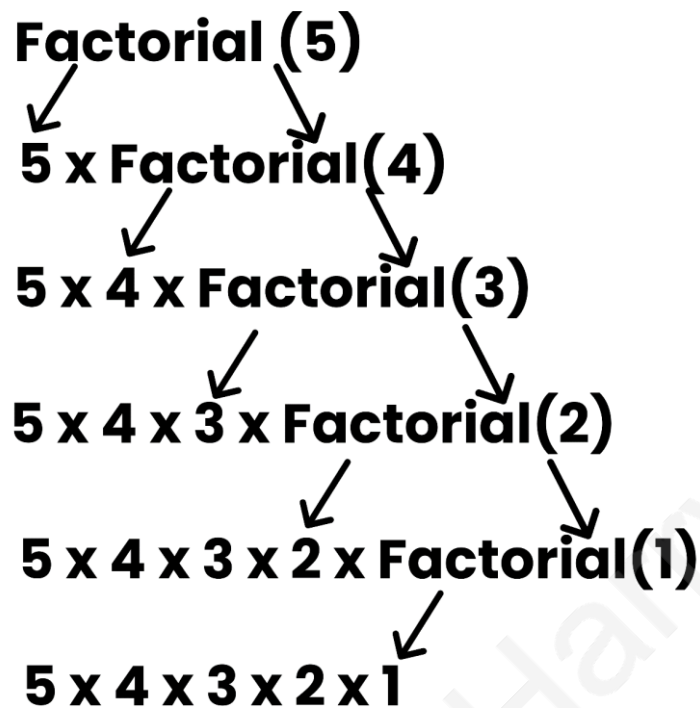
$\text{Factorial}(n) = 1 \times 2 \times 3 \dots \times n$

$\text{Factorial}(n) = 1 \times 2 \times 3 \dots (n-1) \times n$

$\text{Factorial}(n) = \text{Factorial}(n-1) \times n$

Since we can write factorial of a number in terms of itself, we can program it using recursion.

```
int factorial(int x) {
    int f;
    if (x == 0 || x == 1) {
        return 1; // a program to calculate factorial using recursion
    } else {
        f = x * factorial(x - 1);
        return f;
    }
}
```

IMPORTANT NOTES:

1. **Recursion is often a direct way to implement certain algorithms, but not always the most direct for every algorithm.** Recursion is particularly suited for problems that can be divided into smaller, similar subproblems (like factorial computation or tree traversal), but for some algorithms, iterative approaches might be more straightforward or efficient.
2. **The condition in a recursive function that stops further recursion is called the base case.** This correction clarifies that the base case is crucial as it prevents infinite recursion and ensures the function terminates correctly.
3. **Sometimes, due to an oversight by the programmer, a recursive function can continue to run indefinitely without reaching a base case, potentially causing a stack overflow or memory error.** This statement highlights the risk of infinite recursion and its consequences, emphasizing the importance of properly defining base cases in recursive functions.

CHAPTER 5 – PRACTICE SET

1. Write a program using function to find average of three numbers.
2. Write a function to convert Celsius temperature into Fahrenheit.
3. Write a function to calculate force of attraction on a body of mass 'm' exerted by earth. Consider $g = 9.8\text{m/s}^2$.
4. Write a program using recursion to calculate n^{th} element of Fibonacci series.
5. What will the following line produce in a C program:

```
int a = 4;  
printf("%d %d %d \n", a, ++a, a++);
```

6. Write a recursive function to calculate the sum of first 'n' natural numbers.
7. Write a program using function to print the following pattern (first n lines)

*

* * *

* * * * *

CHAPTER 6- POINTERS

A pointer is a variable which stores the address of another variable.



THE “ADDRESS OF” (&) OPERATOR

The address of operator is used to obtain the address of a given variable.

If you refer to the diagrams above,

$\&i \rightarrow 87994$

$\&j \rightarrow 87998$

Format specifier for printing pointer address is ‘%p’.

THE ‘VALUE AT ADDRESS’ OPERATOR (*)

The value at address or * operator is used to obtain the value present at a given memory address. It is denoted by *.

$*(\&i) = 72$

$*(\&j) = 87994$

HOW TO DECLARE A POINTER?

A pointer is declared using the following syntax.

- `int *j =>` declare a variable j of type int-pointer
- `j=&i =>` store address of i in j.

Just like pointer of type integer, we also have pointers to char, float etc.

```
int *in_ptr; //pointer to integer
char *ch_ptr; //pointer to character
float *fl_ptr; //pointer to float
```

Although it's a good practice to use meaningful variable names, we should be very careful while reading and working on programs from fellow programmers.

A PROGRAM TO DEMONSTRATE POINTERS

```
#include <stdio.h>

int main (){
    int i = 8;
    int *j;
    j = &i;
    printf("add i= %u\n",&i);
    printf("add i= %u\n",j);
    printf("add j= %u\n",&j);
    printf("value i= %d\n",i);
    printf("value i= %d\n",*(&i));
    printf("value i= %d\n",*j);
    return 0;
}
```

OUTPUT:

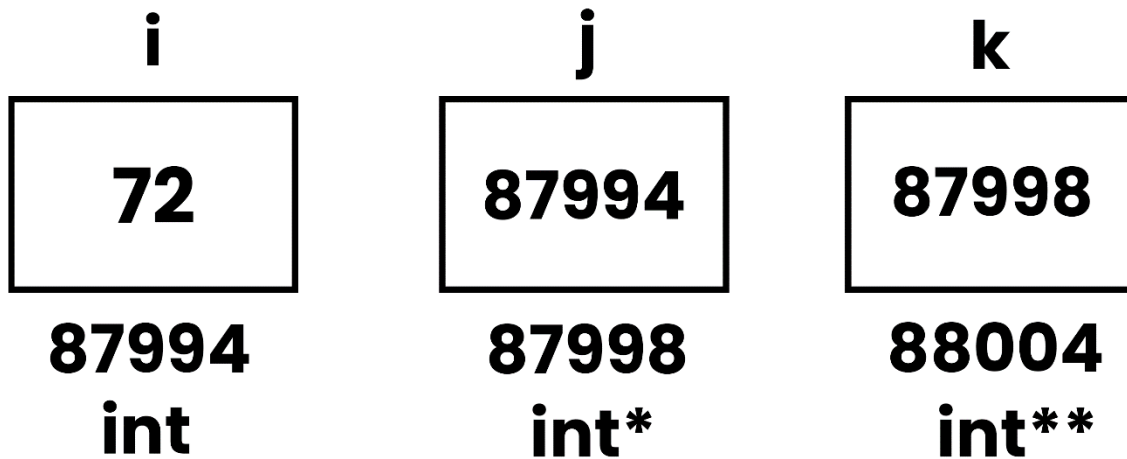
```
add i= 87994
add i= 87994
add j= 87998
value i= 8
value i= 8
value i= 8
```

This program sums it all. If you understand it, you have got the idea of pointers.

POINTER TO A POINTER

Just like 'j' is pointing to 'i' or storing the address of 'i', we can have another variable k which can further store the address of 'j'. What will be the type of 'k'?

```
int **k;
k = &j;
```



We can even go further one level and create a variable 'l' of type int*** to store the address of 'k'. We mostly use int* and int** sometimes in real world programs.

TYPES OF FUNCTION CALL

Based on the way we pass arguments to the function, function calls are of two types.

1. Call by value → Sending the values of arguments.
2. Call by reference → Sending the address of arguments.

CALL BY VALUE

Here the values of the arguments are passed to the function. Consider this example:

```
int c = sum (3,4); //assume x=3 and y=4
```

If sum is defined as sum (int a, int b), the values 3 and 4 are copied to a and b. Now even if we change a and b, nothing happens to the variables x and y.

This is call by value.

In C we usually make a call by value.

CALL BY REFERENCE

Here the address of the variables is passed to the function as arguments.

Now since the addresses are passed to the function, the function can now modify the value of a variable in calling function using * and & operators.

EXAMPLE

```
void swap (int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

This function is capable of swapping the values passed to it. If $a = 3$ and $b = 4$ before a call to `swap(a, b)`, then $a = 4$ and $b = 3$ after calling `swap`.

```
int main(){
    int a = 3;
    int b = 4; // a is 3 and b is 4
    swap(&a, &b);
    return 0; //now a is 4 and b is 3
}
```

CHAPTER 6 – PRACTICE SET

1. Write a program to print the address of a variable. Use this address to get the value of the variable.
2. Write a program having a variable 'i'. Print the address of 'i'. Pass this variable to a function and print its address. Are these addresses same? Why?
3. Write a program to change the value of a variable to ten times of its current value.
4. Write a function and pass the value by reference.
5. Write a program using a function which calculates the sum and average of two numbers. Use pointers and print the values of sum and average in main().
6. Write a program to print the value of a variable i by using "pointer to pointer" type of variable.
7. Try problem 3 using call by value and verify that it does not change the value of the said variable.

CHAPTER 7 – ARRAYS

An array is a collection of similar elements. Array allows a single variable to store multiple values.

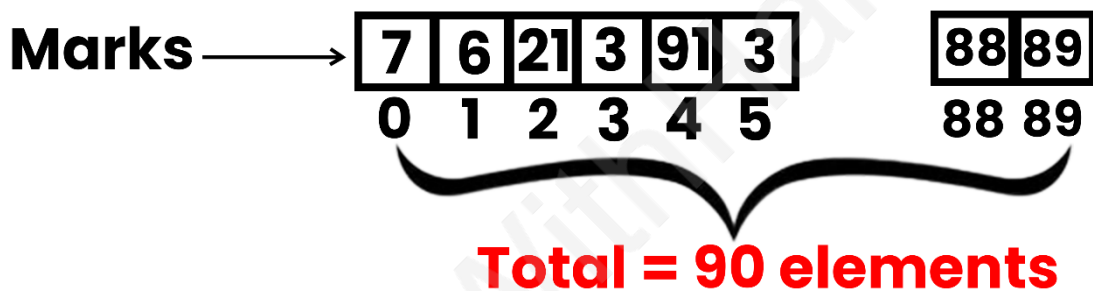
SYNTAX:

```
int marks[90];           // integer array
char name[20];           // character array or string
float percentile[90];    // float array
```

The values can now be assigned to make array like this:

```
marks[0] = 33;
marks[1] = 12;
```

Note: It is very important to note that the array index starts with 0.



ACCESSING ELEMENTS

Elements of an array can be accessed using:

```
scanf("%d", &marks[0]); // input first value
printf("%d", marks[0]); // output first value of the array
```

Quick Quiz: Write a program to accept marks of five students in an array and print them on the screen.

INITIALIZATION OF AN ARRAY

There are many other ways in which an array can be initialized.

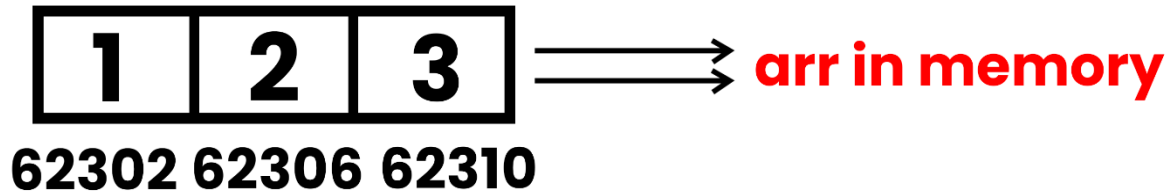
```
int cgpa[3] = {9, 8, 8}; // arrays can be initialized while declaration
float marks[] = {33, 40};
```


ARRAYS IN MEMORY

Consider this array:

```
int arr[3] = {1, 2, 3} // 1 integer = 4 bytes
```

This will reserve $4 \times 3 = 12$ bytes in memory (4 bytes for each integer).



POINTER ARITHMETIC

A pointer can be incremented to point to the next memory location of that type.

Consider this example:

```
int i = 32;
int *a = &i; // a = 87994
a++;          // address of i or value of a = 87998

char a = 'A';
char *b = &a; // a = 87994
b++;          // now a = 87995

float i = 1.7;
float *a = &i; // now a = 87994
a++;          // now a = 87998
```

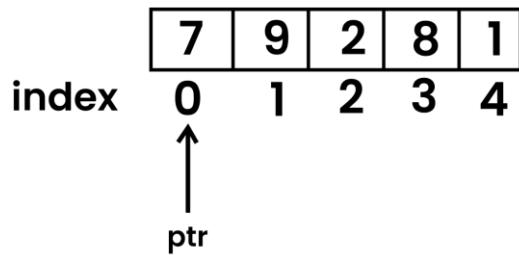
Following operations can be performed on a pointer:

1. Addition of a number to a pointer.
2. Subtraction of a number from a pointer.
3. Subtraction of one pointer from another.
4. Comparison of two pointer variables.

Quick Quiz: Try these operations on another variable by creating pointers in a separate program. Demonstrate all the four operations.

ACCESSING ARRAY USING POINTERS

Consider this array:



If ptr points to index 0, ptr++ will point to index 1 & so on...

This way we can have an integer pointer pointing to first element of the array like this:

```
int *ptr = &arr[0]; // or simple arr
ptr++;
*ptr // will have 9 as its value
```

PASSING ARRAY TO FUNCTIONS

Array can be passed to the functions like this:

```
printArray(arr, n); // function call
void printArray(int *i, int n); // function prototype
// or
void printArray(int i[], int n);
```

MULTIDIMENSIONAL ARRAYS

An array can be of 2 dimension/ 3 dimension/ n dimensions.

A 2 dimensions array can be defined like this:

```
int arr[3][2] = {{1, 4}
                 {7, 9}
                 {11, 22}};
```

We can access the elements of this array as arr[0][0] , arr[0][1] & so on ...

2-D ARRAYS IN MEMORY

A 2d array like a 1d array is stored in contiguous memory blocks like this:

`arr[0][0] arr[0][1] ...`

1	4	7	9	11	22
---	---	---	---	----	----

87224 87228 ..

Quick Quiz: Create a 2-d array by taking input from the user. Write a display function to print the content of this 2-d array on the screen.

CodeWithHarry

CHAPTER 7 – PRACTICE SET

1. Create an array of 10 numbers. Verify using pointer arithmetic that $(ptr+2)$ points to the third element where ptr is a pointer pointing to the first element of the array.
2. If $S[3]$ is a 1-D array of integers then $*(S+3)$ refers to the third element:
 - (i) True.
 - (ii) False.
 - (iii) Depends.
3. Write a program to create an array of 10 integers and store multiplication table of 5 in it.
4. Repeat problem 3 for a general input provided by the user using `scanf`.
5. Write a program containing a function which reverses the array passed to it.
6. Write a program containing functions which counts the number of positive integers in an array.
7. Create an array of size 3×10 containing multiplication tables of the numbers 2, 7 and 9 respectively.
8. Repeat problem 7 for a custom input given by the user.
9. Create a three-dimensional array and print the address of its elements in increasing order.

CHAPTER 8 – STRINGS

A string is a 1-D character array terminated by a null character ('\0')

A null character is used to denote the termination of a string. Characters are stored in contiguous memory locations.

INITIALIZING STRINGS

Since string is an array of characters, it can be initialized as follows:

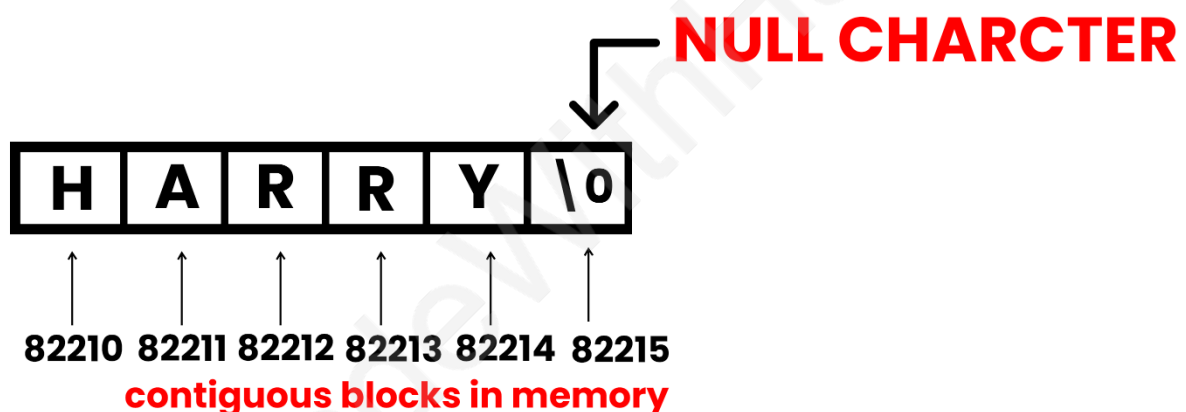
```
char s[] = {'H', 'A', 'R', 'R', 'Y', '\0'};
```

There is another shortcut for initializing string in C language:

```
char s[] = "HARRY"; // In this case C adds a null character automatically.
```

STRINGS IN MEMORY

A string is stored just like an array in the memory as shown below.



Quick Quiz: Create a string using double quotes and print its content using a loop.

PRINTING STRINGS

A string can be printed character by character using printf and %c.

But there is another convenient way to print strings in C.

```
char st[] = "HARRY";  
printf("%s", st); // print the entire string.
```

TAKING STRING INPUT FROM THE USER

We can use %s with scanf to take string input from the user:

```
char st[50];  
scanf ("%s", st);
```

scanf automatically adds a null character when the enter key is pressed.

Note:

1. The string should be short enough to fit into the array.
2. scanf cannot be used to input multi-word strings with spaces.

GETS() AND PUTS()

gets() is a function which can be used to receive a multi-word string.

```
char st[30];  
gets(st); // The entered string is stored in st!
```

multiple gets() calls will be needed for multiple strings.

Likewise, puts can be used to output a string.

```
puts(st); // Prints the string & places the cursor on the next line
```

DECLARING A STRING USING POINTERS

We can declare strings using pointers.

```
char *ptr = "harry";
```

This tells the compiler to store the string in memory and assigned address is stored in a char pointer.

Note:

1. Once a string is defined using `char st[] = "harry"`, it cannot be reinitialized to something else.
2. A string defined using pointers can be reinitialized.

```
ptr = "Rohan";
```

STANDARD LIBRARY FUNCTIONS FOR STRINGS

C provides a set of standard library functions for string manipulation.

Some of the most commonly used string functions are:

STRLEN()

This function is used to count the number of characters in the string excluding the null ('\\0') characters.

```
int length = strlen(st);
```

These functions are declared under <string.h> header file.

STRCPY()

This function is used to copy the content of second string into first string passed to it.

```
char source[] = "harry";  
char target[30];  
strcpy (target,source); //target now contains "harry"
```

target string should have enough capacity to store the source string.

STRCAT()

This function is used to concatenate two strings.

```
char s1[12] = "hello";  
char s2[] = "harry";  
strcat(s1,s2); // s1 now contains "helloharry" <no space in between>
```

STRCMP()

This function is used to compare two strings. It returns 0 if the strings are equal, a negative value if the first string's mismatching character's ASCII value is less than the second string's corresponding mismatching character, and a positive value otherwise.

```
strcmp("far", "joke"); // Negative value  
strcmp("joke", "far"); // Positive value
```

CHAPTER 8 – PRACTICE SET

1. Which of the following is used to appropriately read a multi-word string.
 1. gets()
 2. puts()
 3. printf()
 4. scanf()
2. Write a program to take string as an input from the user using %c and %s confirm that the strings are equal.
3. Write your own version of strlen function from <string.h>
4. Write a function slice() to slice a string. It should change the original string such that it is now the sliced string. Take 'm' and 'n' as the start and ending position for slice.
5. Write your own version of strcpy function from <string.h>
6. Write a program to encrypt a string by adding 1 to the ascii value of its characters.
7. Write a program to decrypt the string encrypted using encrypt function in problem 6.
8. Write a program to count the occurrence of a given character in a string.
9. Write a program to check whether a given character is present in a string or not.

CHAPTER 9 – STRUCTURES

Array and strings → Similar data (int, float, char).

Structures can hold → Dissimilar data.

A C structure can be created as follows:

```
struct employee
{
    int code; // This declares a new user defined data type!
    float salary;
    char name[10];
}; // semicolon is important
```

We can use this user defined data type as follows:

```
struct employee e1; // creating a structure variable
strcpy(e1.name, "harry");
e1.code = 100;
e1.salary = 71.22;
```

So, a structure in C is a collection of variables of different types under a single name.

Quick Quiz: Write a program to store the details of 3 employees from user defined data. Use the structure declared above.

WHY USE STRUCTURES?

We can create the data types in the employee structure separately but when the number of properties in a structure increases, it becomes difficult for us to create data variables without structures. In a nutshell:

- Structures keep the data organized.
- Structures make data management easy for the programmer.

ARRAY OF STRUCTURES

Just like an array of integers, an array of floats and an array of characters, we can create an array of structures.

```
struct employee facebook[100]; // an array of structures
// we can access the data using:
facebook[0].code = 100;
facebook[1].code = 101;
// And so on
```

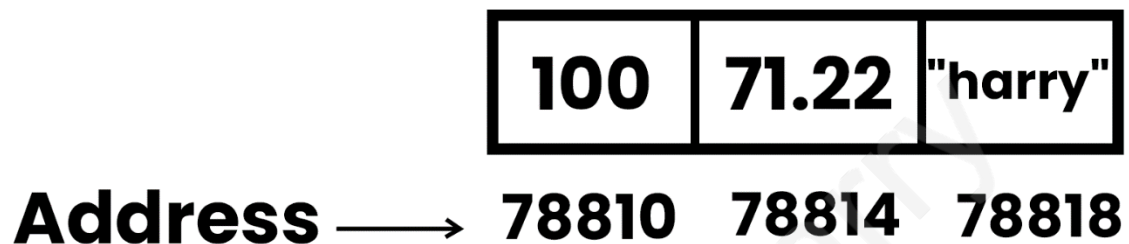
INITIALIZING STRUCTURES

Structures can also be initialized as follows:

```
struct employee harry = {100, 71.22, "harry"};
struct employee shubh = {0}; //All elements set to 0
```

STRUCTURES IN MEMORY

Structures are stored in contiguous memory locations. For the structure 'e1' of type struct employee, memory layout looks like this:



In an array of structures, these employee instances are stored adjacent to each other.

POINTER TO STRUCTURES

A pointer to structures can be created as follows:

```
struct employee *ptr;
ptr = &e1;
// now we can print structure elements using:
printf("%d", (*ptr).code);
```

ARROW OPERATOR

Instead of writing (*ptr).code, we can use arrow operator to access structure properties as follows:

```
(*ptr).code
//or
ptr->code
// here -> is known as the arrow operator.
```

PASSING STRUCTURE TO A FUNCTION

A structure can be passed to a function just like any other data type.

```
void show(struct employee e); // function prototype
```

Quick Quiz: Complete this show function to display the content of employee.

TYPEDEF KEYWORD

We can use the 'typedef' keyword to create an alias name for data types in C.

'typedef' is more commonly used with structures.

```
struct Complex
{
    float real;
    float img; // struct complex c1,c2, for defining complex numbers
};

typedef struct Complex
{
    float real;
    float img; // ComplexNo c1,c2,for defining complex numbers
} ComplexNo;
```

EXAMPLE USAGE

Using the typedef alias, you can declare complex number variables more succinctly:

```
ComplexNo c1, c2;
```

CHAPTER 9 – PRACTICE SET

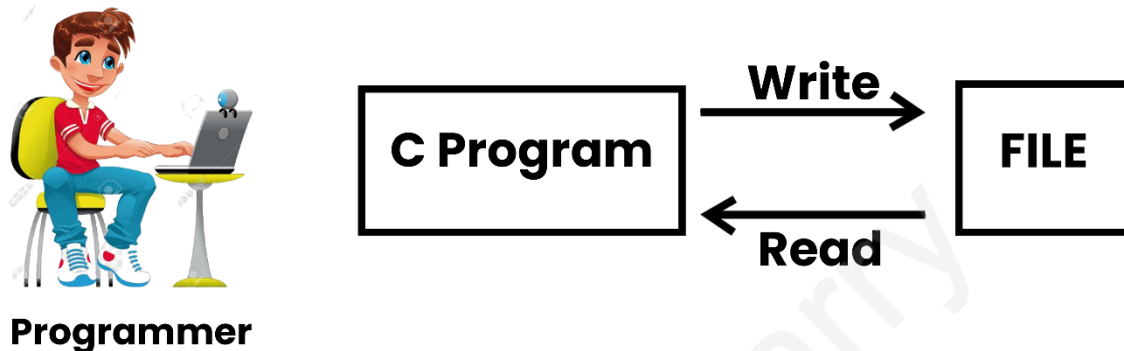
1. Create a two-dimensional vector using structures in C.
2. Write a function 'sumVector' which returns the sum of two vectors passed to it. The vectors must be two-dimensional.
3. Twenty integers are to be stored in memory. What will you prefer- Array or structure?
4. Write a program to illustrate the use of arrow operator $\rightarrow$ in C.
5. Write a program with a structure representing a complex number.
6. Create an array of 5 complex numbers created in Problem 5 and display them with the help of a display function. The values must be taken as an input from the user.
7. Write problem 5's structure using 'typedef' keywords.
8. Create a structure representing a bank account of a customer. What fields did you use and why?
9. Write a structure capable of storing date. Write a function to compare those dates.
10. Solve problem 9 for time using 'typedef' keyword.

CHAPTER 10 – FILE I/O

The random-access memory is volatile, and its content is lost once the program terminates. In order to persist the data forever we use files.

A file is data stored in a storage device.

A C program can talk to the file by reading content from it and writing content to it.



FILE POINTER

A “FILE” is a structure which needs to be created for opening the file.

A file pointer is a pointer to this structure of the file.

(FILE pointer is needed for communication between the file and the program).

A FILE pointer can be created as follows:

```
FILE *ptr;  
ptr = fopen("filename.ext"; "mode");
```

FILE OPENING MODES IN C

C offers the programmers to select a mode for opening a file.

Following modes are primarily used in C File I/O.

```
"r"    -> open for reading  
"rb"   -> open for reading in binary  
"w"    -> open for writing // If the file exists, the contents will be  
overwritten  
"wb"   -> open for writing in binary  
"a"    -> open for append // If the file does not exist, it will be created
```

TYPES OF FILES

Primarily, there are two types of files:

1. Text files (.txt, .c)
2. Binary files (.jpg, .dat)

READING A FILE

A file can be opened for reading as follows:

```
FILE *ptr;  
ptr = fopen("harry.txt", "r");  
int num;
```

Let us assume that "harry.txt" contains an integer we can read that integer using:

```
fscanf(ptr, "%d", &num); // fscanf is file counterpart of scanf
```

This will read an integer from file in Num variables.

Quick Quiz: Modify the program above to check whether the file exists or not before opening the file.

CLOSING THE FILE

It is very important to close the file after read or write. This is achieved using fclose as follows:

```
fclose(ptr);
```

This will tell the compiler that we are done working with this file and the associated resources could be freed.

WRITE TO A FILE

We can write to a file in a very similar manner like we read the file

```
FILE *fptr;  
fptr = fopen("harry.txt", "w");  
int num = 432;  
fprintf(fptr, "%d", num);  
fclose(fptr);
```

FGETC() AND FPUTC()

fgetc and fputc are used to read and write a character from / to a file.

```
fgetc(ptr);           // used to read a character from file
fputc('c', ptr);      // used to write character 'c' to the file
```

EOF : END OF FILE

fgetc returns EOF when all the characters from a file have been read. So, we can write a check like below to detect end of file:

```
while(1)
{
    ch = fgetc(ptr); // when all the content of a file has been read break
the loop!
    if (ch == EOF)
    {
        break;
    }
    // code
}
```

CHAPTER 10 – PRACTICE SET

1. Write a program to read three integers from a file.
2. Write a program to generate multiplication table of a given number in text format. Make sure that the file is readable and well formatted.
3. Write a program to read a text file character by character and write its content twice in separate file.
4. Take name and salary of two employees as input from the user and write them to a text file in the following format:
 - i. Name1, 3300
 - ii. Name2, 7700
5. Write a program to modify a file containing an integer to double its value.

PROJECT 2: SNAKE, WATER, GUN

Snake, water, gun or rock, paper, scissors is a game most of us have played during school time. (I sometimes play it even now).

Write a C program capable of playing this game with you.

Your program should be able to print the result after you choose snake/water or gun.

CodeWithHarry

CHAPTER 11 – DYNAMIC MEMORY ALLOCATION

C is a language with some fixed rules of programming. For example: Changing the size of an array is not allowed.

DYNAMIC MEMORY ALLOCATION

Dynamic memory allocation is a way to allocate memory to a data structure during the runtime. We can use DMA function available in C to allocate and free memory during runtime.

FUNCTION FOR DMA IN C

Following function are available in C to perform dynamic memory allocation:

1. malloc()
2. calloc()
3. free()
4. realloc()

MALLOC() FUNCTION

malloc stands for memory allocation. It takes number of bytes to be allocated as an input and returns a pointer of type void.

Syntax:

```
ptr = (int*)malloc(30* sizeof (int))
```

The expression returns a null pointer if the memory cannot be allocated.

Quick Quiz: Write a program to create a dynamic array of 5 floats using malloc().

CALLOC() FUNCTION

calloc stands for continuous allocation. It initializes each memory block with a default value of 0.

Syntax:

```
ptr = (float*)calloc(30, sizeof (float));  
//allocates contiguous space in memory for 30 blocks (floats)
```

If the space is not sufficient, memory allocation fails, and a NULL pointer is returned.

Quick Quiz: Write a program to create an array of size n using calloc where n is an integer entered by the user.

FREE() FUNCTION

We can use free() function to deallocate the memory. The memory allocated using calloc/malloc is not deallocated automatically.

Syntax:

```
free(ptr); //memory of ptr is released.
```

Quick Quiz: Write a program to demonstrate the usage of free() with malloc().

REALLOC() FUNCTION

Sometimes the dynamically allocated memory is insufficient or more than required. realloc is used to allocate memory of new size using the previous pointer and size.

Syntax:

```
ptr = realloc (ptr, newsize);  
ptr = realloc (ptr, 3*sizeof(int));
```

CHAPTER 11 – PRACTICE SET

1. Write a program to dynamically create an array of size 6 capable of storing 6 integers.
2. Use the array in problem 1 to store 6 integers entered by the user.
3. Solve problem 1 using `calloc()`.
4. Create an array dynamically capable of storing 5 integers. Now use `realloc` so that it can now store 10 integers.
5. Create an array of multiplication table of 7 upto 10 ($7 \times 10 = 70$). Use `realloc` to make it store 15 number (from 7×1 to 7×15).
6. Attempt problem 4 using `calloc()`.

CodeWithHarry